

It's easy!

LET'S LEARN ABOUT:

Activation Functions

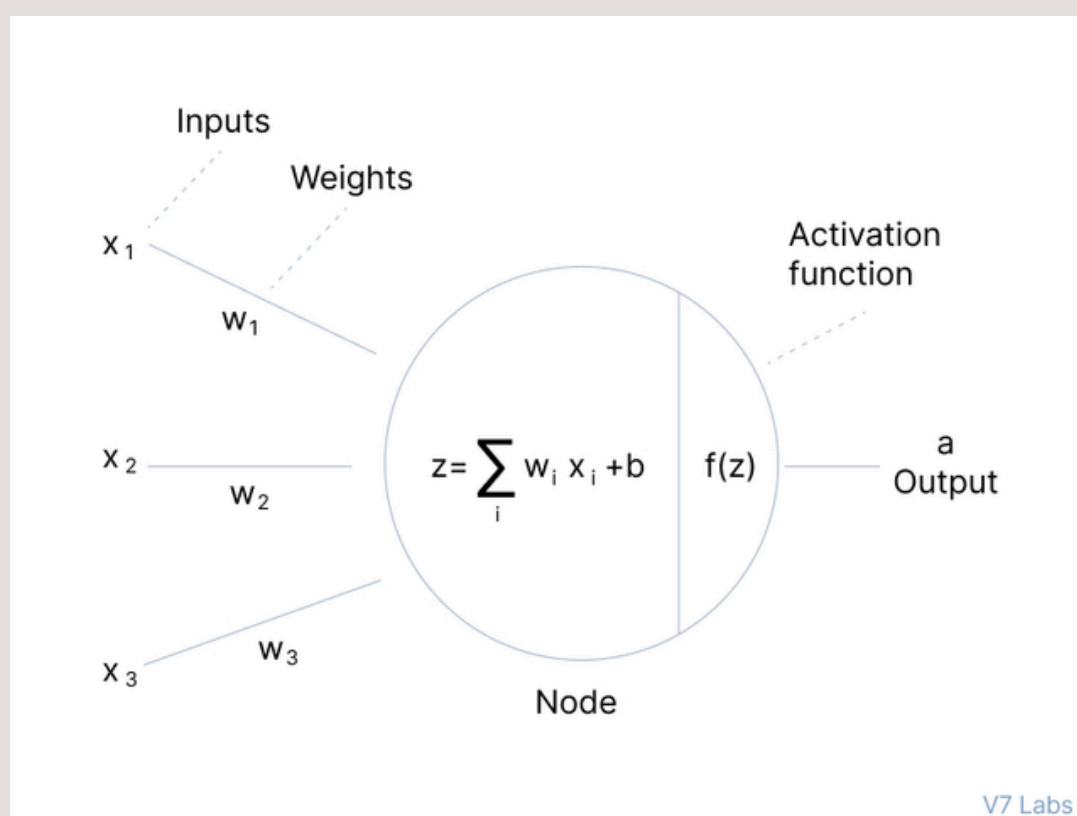
Let's answer three simple Questions
What, How & Where

A series of posts purely based on ML, DL and LLM's

Presented by:
Sumanth Pagadala

What are Activation Functions?

Activation functions are simple mathematical operations (I mean it!) applied to the output of a neuron in a neural network to introduce non-linearity into the model, allowing it to capture complex patterns in the data.



This article will be talking about the $f(z)$ from the diagram

Why do we need A-F's?

Without activation functions, neural networks would behave like linear regression models, limiting their ability to solve more advanced tasks.



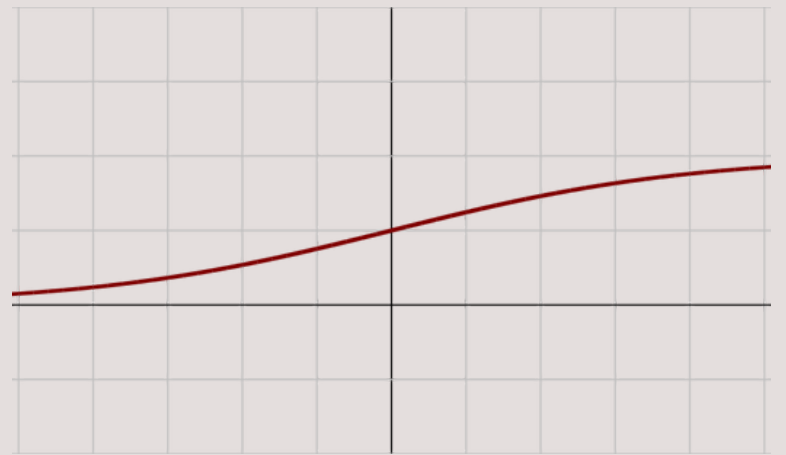
Types Activation Functions

Sigmoid

What: The sigmoid function squashes the input to a value between 0 and 1.

How:

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



Where: Sigmoid is commonly used in binary classification tasks. For example, in medical diagnosis where we need to classify whether a tumor is malignant or benign, the output of the neural network is interpreted as a probability (e.g., 0.8 could mean an 80% chance the tumor is malignant). Sigmoid is ideal here since it outputs a value between 0 and 1, representing the likelihood.

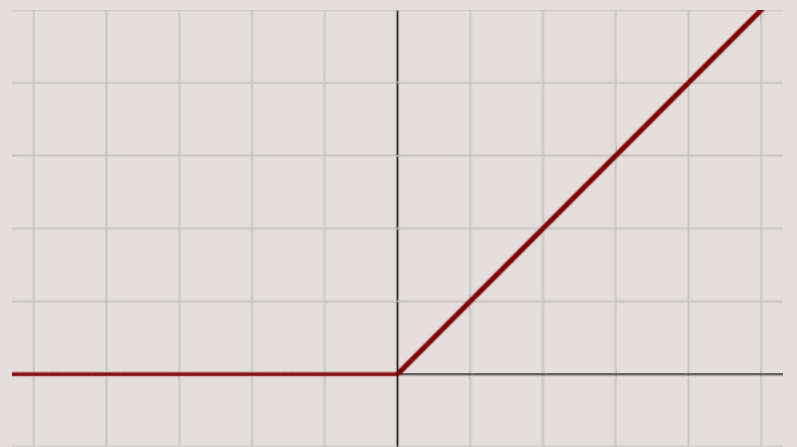
Types Activation Functions

ReLU (Rectified Linear Unit)

What: ReLU is one of the most commonly used activation functions. It outputs the input directly if positive; otherwise, it outputs zero.

How:

$$\text{ReLU}(x) = \max(0, x)$$



Where: ReLU is used extensively in deep learning models, especially in hidden layers of convolutional neural networks (CNNs). For instance, in image classification tasks like recognizing handwritten digits (MNIST dataset), ReLU helps activate neurons when relevant features (like edges in an image) are detected, speeding up learning and helping the model capture complex image structures.

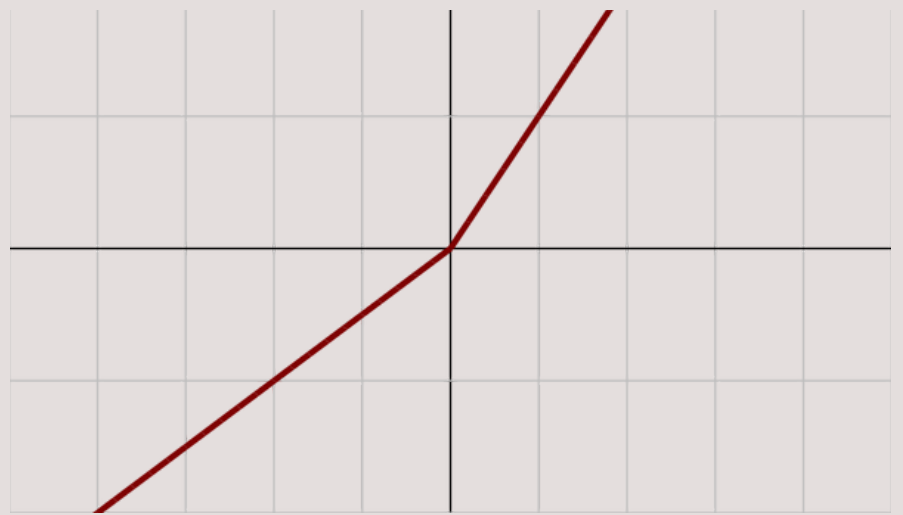
Types Activation Functions

Leaky ReLU

What: Leaky ReLU allows a small, non-zero gradient for negative input values, unlike ReLU.

How:

$$\text{Leaky ReLU}(x) = \max(0.01x, x)$$



Where: Leaky ReLU is useful in deep networks where ReLU might cause some neurons to become inactive (dead neurons). For instance, in a deep learning model analyzing customer behavior data (predicting churn), Leaky ReLU can prevent neurons from getting "stuck" at zero, allowing the model to continue learning even with small negative inputs, which might represent unusual or rare patterns in the data.

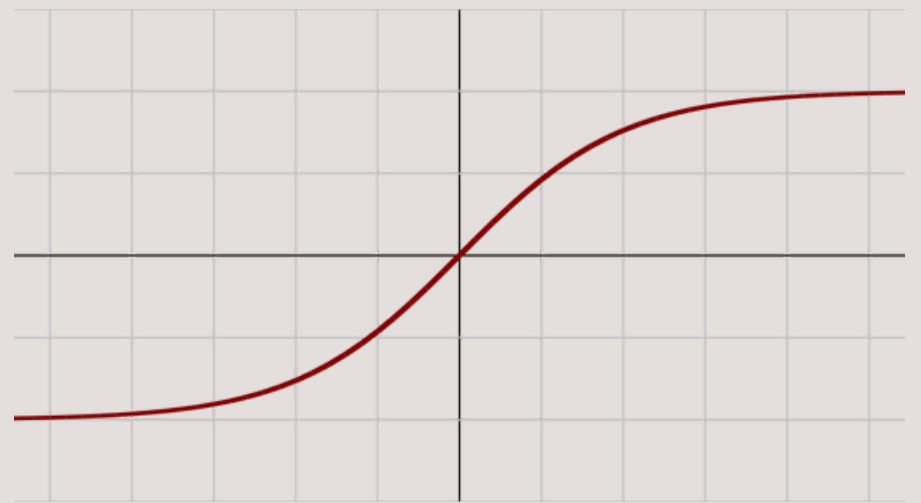
Types Activation Functions

Tanh

What: Tanh squashes the input to a range between -1 and 1.

How:

$$\text{Tanh}(x) = \frac{2}{1 + e^{-2x}} - 1$$



Where: Tanh is often used in Recurrent Neural Networks (RNNs) for time-series predictions, where the network needs to consider past values. For example, in predicting stock prices, Tanh helps by providing a symmetric output range (centered around zero) to balance positive and negative price changes, which can make the network more stable during training.

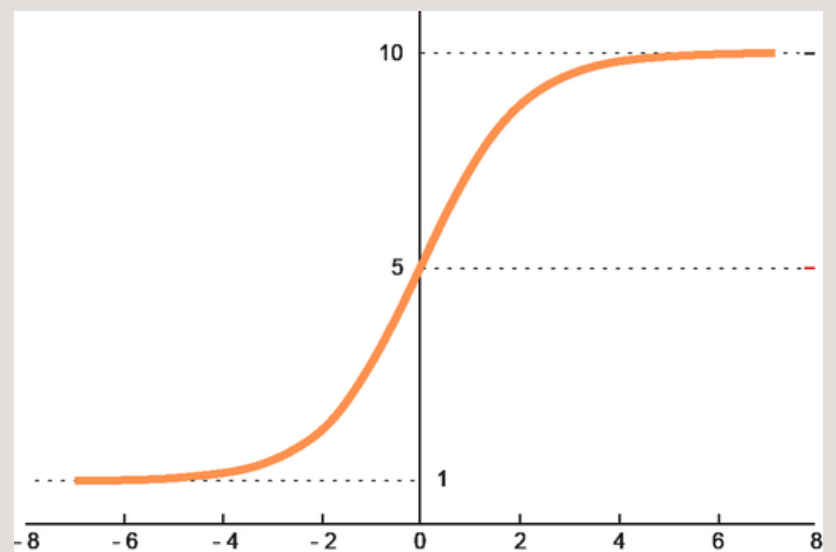
Types Activation Functions

Softmax

What: Softmax converts the output into a probability distribution across multiple classes.

How:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$



Where: Softmax is primarily used in the output layer for multi-class classification tasks. For example, in classifying types of flowers (like in the Iris dataset, where the goal is to predict whether a flower is Setosa, Versicolor, or Virginica), Softmax converts the network's output into probabilities for each flower type, ensuring the probabilities across all classes sum to 100%, making it ideal for multi-class problems.

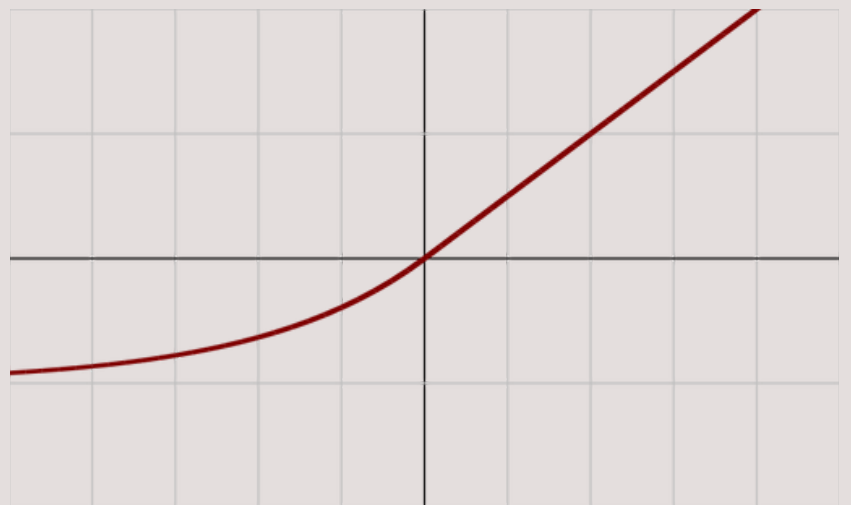
Types Activation Functions

ELU (Exponential Linear Unit)

What: ELU is like ReLU but smoother for negative values, which helps the network learn better. It allows small negative outputs rather than zero.

How:

$$\text{ELU}(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha(e^x - 1), & \text{if } x \leq 0 \end{cases}$$



Where: ELU is useful in deep networks to avoid issues like vanishing gradients. For example, in medical image segmentation, where pixel-level precision is needed (like identifying the boundaries of tumors), ELU can help the network learn more effectively by handling negative values smoothly, which improves convergence during training.

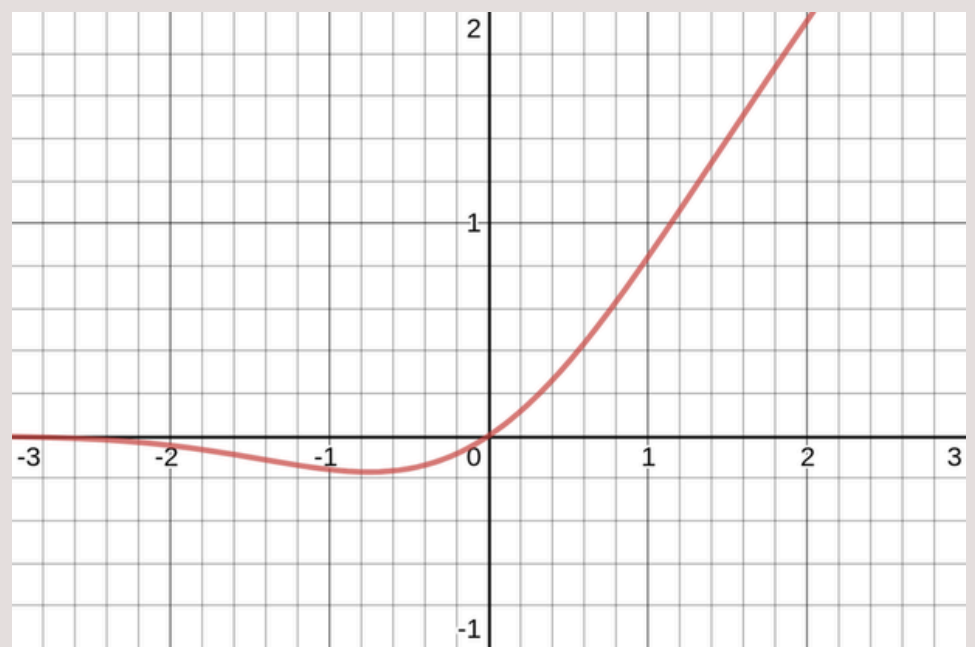
Types Activation Functions

GELU (Gaussian Error Linear Unit)

What: GELU combines aspects of both ReLU and Sigmoid, applying a smooth activation that considers the input's value.

How:

$$\text{GELU}(x) = x \cdot \Phi(x)$$



Where: GELU is used in cutting-edge models like the Transformer architecture, which powers language models like GPT. In natural language processing tasks, GELU helps improve the network's ability to capture complex relationships in data, such as in machine translation or text summarization, by applying a smoother non-linearity than ReLU.

Example

Let's take a quick example to understand the activation function.

Consider two inputs from the titanic dataset:

Ticket Price: \$50

Gender: Female (we'll encode this as 1, where 1 = female and 0 = male)

Assume we have a simple neural network model with weights for these two inputs and a bias term. Let's say the weights for ticket price and gender are 0.01 and 0.5 respectively, and the bias is -1.

$$z = (w_1 \times \text{Ticket Price}) + (w_2 \times \text{Gender}) + b$$

Let's consider $w_1 = 0.01$, $w_2 = 0.5$, bias = -1

$$z = (0.01 \times 50) + (0.5 \times 1) + (-1) = 0.5 + 0.5 - 1 = 0$$

Note: The above values are just for your understanding, follow the procedure



Example

Now that we have the weighted sum $z = 0$, we'll apply the sigmoid function to convert this into a probability.

The Sigmoid function is:

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

For $z = 0$, the calculation is:

$$\text{Sigmoid}(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

The sigmoid function outputs 0.5, meaning that based on the ticket price and gender alone, the model predicts a 50% chance that this person survived the Titanic disaster.

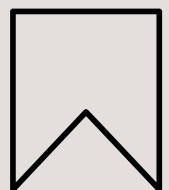
Remember, as we discussed earlier, weights and biases are trainable parameters. When you train the model on more data, these parameters get tuned during backpropagation (next topic).



It's easy!

Congratulations!

You learnt a new topic today!



Save the post and follow me for more content on ML,DL and LLM's

SUMANTH PAGADALA
